

Chapter #3:

Parser (Syntax Analysis)

Q. What is parser?

- ⇒ Parsing is the second stage of the Compiler's front end.
- ⇒ The parser works with the program as transformed by the scanner.
- ⇒ It sees a stream of words where each word is annotated with a syntactic category.
- ⇒ The parser derives a syntactic structure of the program, fitting the words into a grammatical model of the source programming language.
- ⇒ If the parser determines that the input stream is a valid program, then the parser attempts to build a derivation for the i/p program used by the later phases of compilation.
- ⇒ If the input stream is not a valid program, the parser reports the problem and appropriate diagnostic information to the user.

Main task of parser:

⇒ The primary purpose of the parser is to determine whether or not the i/p program is a syntactically valid sentence in the source program.

⇒ Before building the parser, both formal mechanism for specifying the syntax of the source language and a systematic method of determining membership in this formally specified language.

⇒ By restricting the form of the source language to a set of efficiently answerable languages called CFG, ensure that the parser can efficiently answer the membership questions.

⇒ CFG as a notation for specifying the syntax.

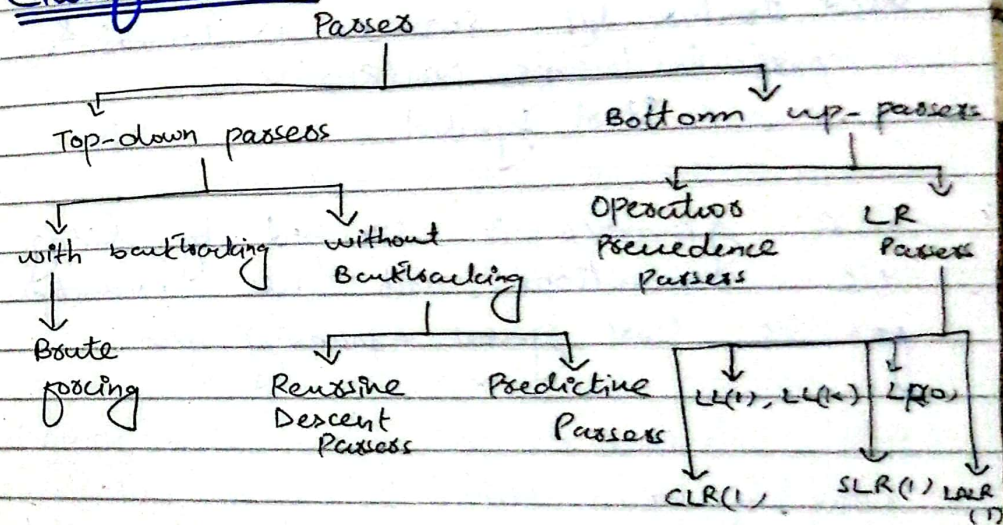
⇒ Many algorithms have been proposed to answer the membership question for CFG.

Types of parsing algorithms

⇒ Parsing algorithms fall into two general categories.

- Top-down parser
- Bottom-up parser

Classification:



Grammars

- ⇒ A grammar consists of 4 components:
- set of Tokens (Terminals)
 - set of Non-terminals
 - set of productions
- ⇒ it is basically a set of rules that defines the valid sentence by sequence of terminals & non-terminals.
- ⇓ each production has a non-terminal, followed by arrow, followed by sequence of terminals & non-terminals.
- particular language: NT (RHS),
 - Start symbol

⇒ A grammar is specified by listing the productions, with the production for the start symbol appearing first.

eg expression with single digits having either + or - b/w them

$L \rightarrow D + D \mid D - D \mid D$

$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Q. How to derive strings from the grammar?

⇒ Start with the production having the start symbol on the left hand side

⇒ Repeatedly replace all the non-terminals by their productions.

Sentence:

⇒ A string of symbols that can be derived from the rules of a grammar.

Production:

⇒ Each rule in CFG is called production.

Non-terminal symbol:

⇒ A syntactic variable used in a grammar's productions called non-terminal symbols.

Terminal symbol:-

⇒ A word that can occur in a sentence.

⇒ A word consists of a lexeme

and its syntactic category words are represented in a grammar using their syntactic category.

Derivation:

⇒ A sequence of rewriting steps that begins with the grammar start symbol and ends with a sentence in the language.

Sentential form:

⇒ In derivation process, sentential form is a collection of terminal or non-terminal symbols.

A string is called a sentential form if it occurs in some step of a valid derivation.

Any sentential form can be derived from the start symbol in zero or more steps.

∴ lock up conditions

Q. what is the purpose of CFG?

⇒ Grammars are used to systematically describe the syntax of programming language constructs like expressions and statements.

⇒ Using a syntactic variable stmt to denote statements and variable exps to denote expressions, the production $stmt \rightarrow if (exps) stmt \text{ else } stmt$ specifies the structure of this form of conditional statements.

write CFG for simple arithmetic expressions?

expression $\rightarrow$ expression + term

expression $\rightarrow$ expression - term

expression $\rightarrow$ term

term $\rightarrow$ factor / factor

term $\rightarrow$ factor

factor $\rightarrow$ (expression)

factor $\rightarrow$ id

TS are: id, +, -, *, /, (,)

NTS are: expression, term & factor
start symbol: expression is the start symbol

Notation Conventions

⇒ The following notational conventions for grammars will be used:

⇒ (1) These symbols are **Terminals**:

(a) Lowercase letters early in the alphabet, such as a, b, c.

(b) Operator symbols such as +, *, and so on.

(c) Punctuation symbols such as parentheses, comma and so on.

(d) Boldface strings such as **if**, each of which represent a single terminal symbol.

2) These symbols are **non-terminals**:

(a) Uppercase letters (A, B, ... etc)

(b) The letter S as start symbol.

(c) Lowercase italic non-terminals such as *a*, *b*, etc.

What is derivations?

⇒ The process of deriving strings of terminals and non-terminals from the start variable by applying some set of productions is called a derivation.

Derivation is denoted by $\rightarrow$.
⇒ we may extend the $\rightarrow$ relationship to represent zero, one or many derivations steps.

⇒ we use * to denote "zero or more steps"

⇒ we use + to denote "one or more steps"

1-step derivation:

⇒ if a string is obtained by applying only "one step" then it is called 1-step derivation and it is denoted by $\Rightarrow$.

Types of Derivatives

1) LMD:

⇒ The process of obtaining a

String of terminals from a sequence of replacements such that only leftmost non-terminal or variable is replaced at each and every step is called leftmost derivation.

(2) RMD:

⇒ The process of obtaining a string of terminals from a sequence of replacements such that only rightmost non-terminal or variable is replaced at each and every step is called rightmost derivation.

Q. Define Top-down Parser?

⇒ When the parse tree can be constructed from root and expanded to leaves then such type of parser is called Top-down parser. The name itself tells us that the parse tree can be built from top to bottom.

Q. Define bottom-up parser?

⇒ When the parse tree can be constructed from leaves to root, then such type of parser is called as bottom-up parser. Thus the parse tree is built in bottom up manner.

Q. Difference between top down & bottom up parser-

Top-down Parser
(Left most)

⇒ Parse tree can be built from root to leaves.

⇒ This is simple to implement.

⇒ It is applicable to small class of languages.

⇒ Less efficient. Various problems

Bottom up parser
(Right most)

⇒ Parse tree can be built from leaves to root.

⇒ This is complex to implement.

⇒ It is applicable to a broad class of languages.

⇒

that during when the bottom up passes handles up passes handles top down techniques are ambiguous left ambiguous grammars are ambiguity left conflicts occur in parse table.

=> Various Parsing techniques are:

- Recursive descent
- Predictive Parser
- LR Parser
- Operator precedence
- Bottom up

Given Grammars

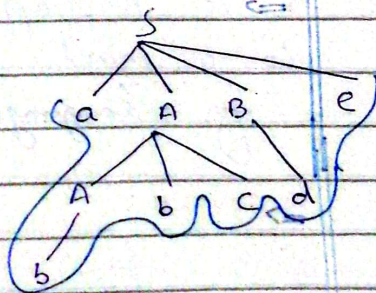
$S \rightarrow aABe$

$A \rightarrow Abc \mid b$

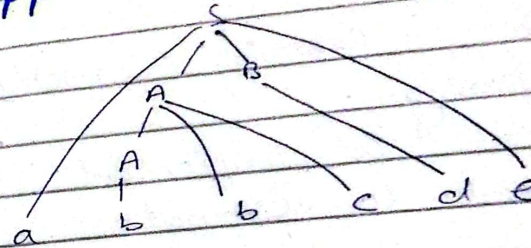
$B \rightarrow d$

String $\rightarrow abbcde$

Construct parse tree from top down passes?



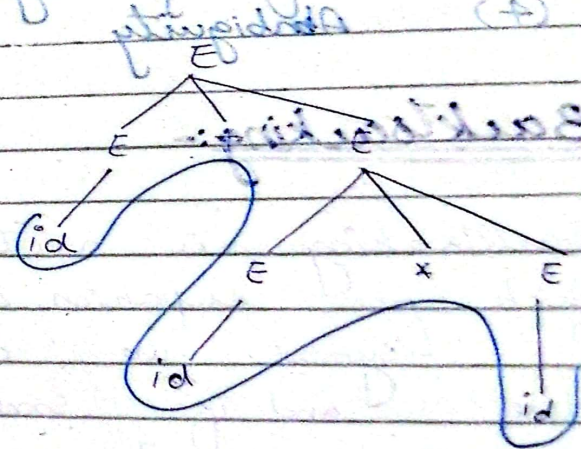
Generate parse tree using bottom up approach abbcde.



Q. Check the given string belongs to the grammar.

G: $E \rightarrow E+E \mid E * E \mid E = E \mid id$

W: $id + id * id - id / id$



Yes, the given string belongs to the grammar.

Q. Problems with Top-down parsing:

⇒ There are certain problems in top-down parsing.

⇒ In order to implement the parsing need to eliminate these problems.

⇒ These problems are:

- (1) Backtracking
- (2) Left-Recursion
- (3) Left-factoring
- (4) Ambiguity

(1) Backtracking:-

⇒ Backtracking is a technique in which for expansion of non-terminal symbol we choose one alternative and if some mismatch occurs then we try another alternative if any.

⇒ Backtracking is a fundamental technique in top-down parsing that enables the handling of ambiguity and the exploration of alternative choices. It allows parser to systematically backtrack to previous decision points and try different production rules when a chosen path fails.

⇒ It is a brute force approach.

↓
To search all possible solution and pick out required solution.

⇒ If matching occurs then construction of parse tree is successful otherwise syntax error.

Short summary:

- When Non-terminal expanding first time then go with the first alternative and
- Compare with the i/p string. If it does not match then continue to match with each and every alternative.

⇒ if the grammar is non-deterministic then we use brute technique and if the grammar is deterministic, then we go with the predictive parser.

Non-deterministic Grammars:-

⇒ The grammar with common prefix is known as non-deterministic grammar.

o $A \rightarrow \alpha\beta / \alpha\beta$

Drawbacks:-

o Brute force requires a lot of backtracking, takes $O(2^n)$

o Back-tracking is very costly & reduces the performance of parser.

o Debugging is very difficult.

⇒ if for a non-terminating grammar, there are multiple production rules beginning with the same terminal symbol then to get the correct derivation we need to try all these alternatives.

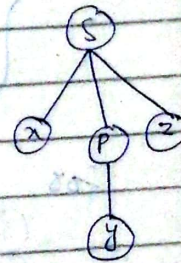
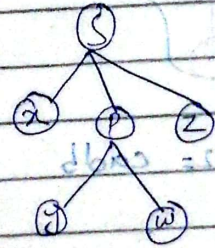
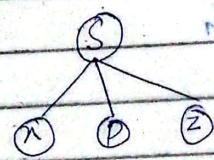
⇒ In backtracking we need to move some levels upward in order to check the possibilities. This increase lot of overhead in implementation of parsing. And hence it becomes necessary to eliminate the backtracking by modifying the grammar.

Example:-

$S \rightarrow xPz$

$P \rightarrow kw / y$

Then $kw = kw$

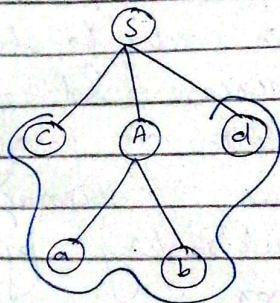


Example-2

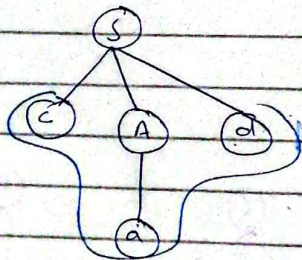
$S \rightarrow cAd$

$A \rightarrow abla$

$w_1 = cad$

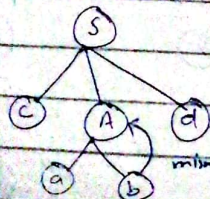


$cabd \neq cad$
mismatch occurs



$cad = cad$
match

for $w_2 = cadb$



mismatch to
backtrack occur

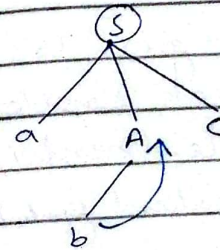
Example-3

$S \rightarrow aAc \mid aB$

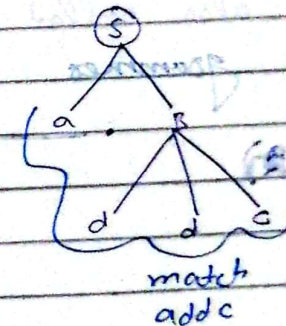
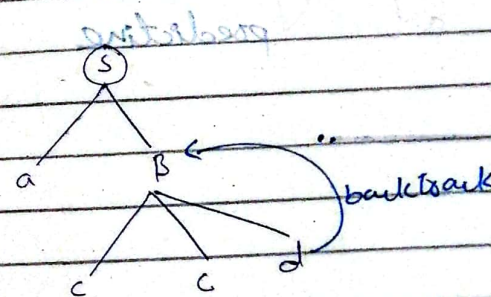
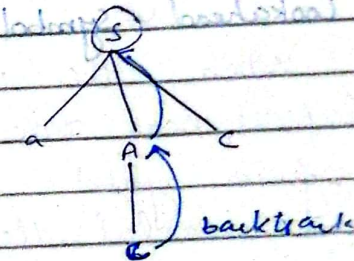
$A \rightarrow b \mid c$

$B \rightarrow cc \mid ddc$

$w = addc$



abc



$\Rightarrow$ TDP may be constructed for both left factor and non-left-factor

Back-track free grammars:

→ A CFG for which the left-most, top-down passes can always predict the correct rule with a lookahead of at most one word.

⇒ Lookahead symbol:

When the parser goes to select the next rule, it can consider the focus symbol and the next i/p symbol, called the lookahead symbol.

⇒ A back-track free grammar is also called a predictive grammar.

2)

2) Left Recursion:

→ Direct LR

→ Indirect LR

($A \rightarrow B a \mid \epsilon$
 $B \rightarrow A b \mid \epsilon$)

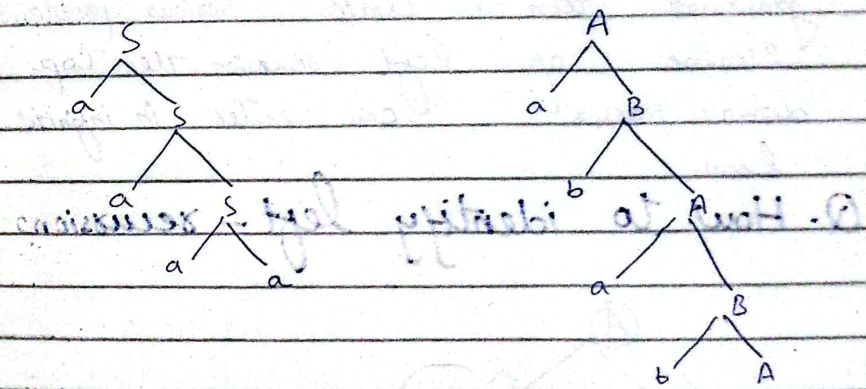
• Grammar G is said to be recursive if it contains productions containing recursion in direct or indirect form.

• Example of direct recursion:
 $S \rightarrow a S a$

∴ Same non-terminal is repeated on right hand side.

Example of indirect recursion:

$A \rightarrow a B a$
 $B \rightarrow b A b$



• Direct recursion can further be classified into:
• left-recursion • Right-recursion

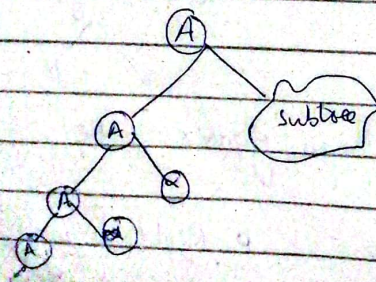
$S \rightarrow aS$ — right recursion
 $S \rightarrow Sa$ — left-recursion
 $S \rightarrow aSa$ — simple recursion
 $S \rightarrow Sas$ — Contains both LR & RR.

$\Rightarrow$ if non-terminal repeat entire left then called left recursion.
 $A \Rightarrow^+ A\alpha$

$\Rightarrow$ means deriving the L.P. in one or more steps. Then A can be non-terminal and α denotes the L.P. string. **why to remove?**

$\Rightarrow$ if left recursion is present in the grammar then it creates serious problems. Because of left recursion the top down parser can enter in infinite loop.

Q. How to identify left-recursion?



A production of grammar is said to have left recursion if the leftmost variable of its R.H.S. is same as variable of its left-recursion.

In terms of function:-

LR	RR
$A()$ $\{$ $A();$ $\alpha;$ $\}$ // infinite loop	$A()$ $\{$ $\alpha;$ $A();$ $\}$

Generalized Structure of left recursion:-

- $A \rightarrow A\alpha | B$
- $A \rightarrow A\alpha | B_1 | B_2$
- $A \rightarrow A\alpha | B_1 | B_2 | \dots | B_n$
- $A \rightarrow A\alpha_1 | A\alpha_2 | B$
- $A \rightarrow A\alpha_1 | A\alpha_2 | \dots | A\alpha_n | B$
- $A \rightarrow A\alpha_1 | A\alpha_2 | \dots | A\alpha_n | B_1 | B_2 | \dots | B_n$

$\hookrightarrow A \rightarrow BA'$
 $A' \rightarrow \alpha A' | \epsilon$

$\Rightarrow A \rightarrow B_1 A' | B_2 A' | B_3 A' | \dots | B_n A'$
 $A' \rightarrow \alpha_1 A' | \alpha_2 A' | \alpha_3 A' | \dots | \alpha_n A' | \epsilon$

Examples

Elimination of left recursion

Let, G be a context free grammar having a production rule with left recursion.

$$A \rightarrow A\alpha$$

$$A \rightarrow \beta$$

Then we eliminate left recursion by re-writing the production rule as:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A'$$

$$A' \rightarrow \epsilon$$

for example:

$$E \rightarrow E+T \mid T$$

By map grammar with rule:

$$A \rightarrow A\alpha \mid \beta$$

$$A = E$$

$$\alpha = +T$$

$$\beta = T$$

Then the rule becomes,

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

2. Eliminate left recursion

$$A \rightarrow A(A) \mid a$$

$$A \rightarrow aA'$$

$$A' \rightarrow (A)A' \mid \epsilon$$

$$A \rightarrow AaA \mid b$$

$$A \rightarrow bA'$$

$$A' \rightarrow aAA' \mid \epsilon$$

$$S \rightarrow Sabc$$

$$S \rightarrow cS'$$

$$S' \rightarrow abs' \mid \epsilon$$

$$S \rightarrow S0S1 \mid 011$$

$$S \rightarrow 0S' \mid 1S'$$

$$S' \rightarrow 0S1S' \mid \epsilon$$

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T*F \mid F$$

$$F \rightarrow (E) \mid id$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

reduction: $33 \times 3 \times 3$

$$A \rightarrow Ba \mid Aa \mid c$$

$$B \rightarrow Bb \mid Ab \mid d$$

- case of indirect left recursion

$$A \rightarrow Ba \mid Aa \mid c$$

$$A \rightarrow Ba \mid cA'$$

$$A' \rightarrow aA' \mid \epsilon$$

New Grammar

$$A \rightarrow Ba \mid cA'$$

$$B \rightarrow Ab \mid d$$

$$A \rightarrow BaA' \mid cA'$$

$$A' \rightarrow aA' \mid \epsilon$$

$$B \rightarrow Bb \mid BaA' \mid d$$

$$CA' \mid d$$

Now:

$$A \rightarrow BaA' \mid cA'$$

$$A' \rightarrow aA' \mid \epsilon$$

$$B \rightarrow CA' \mid Bb' \mid d \mid B'$$

$$B' \rightarrow bB' \mid aA' \mid d \mid B'$$

$$F \rightarrow (E) \mid id$$

$$R \rightarrow R * | R R | (R) | id$$

$$R \rightarrow (R) B' | id R'$$

$$R' \rightarrow * R' | R R' | \epsilon$$

$$S \rightarrow (L) | a$$

$$L \rightarrow L, S | S$$

$$S \rightarrow (L) | a$$

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' | \epsilon$$

$$A \rightarrow ABd | Aa | a$$

$$B \rightarrow Bb | b$$

remove left recursion.

Consider the rule,

$$A \rightarrow ABd | Aa | a$$

Use the rule $A \rightarrow A\alpha | \beta$

$$A \rightarrow aA'$$

$$A' \rightarrow Bd | A' | aA'$$

$$A' \rightarrow \epsilon$$

$$B \rightarrow Bb | b$$

$$B \rightarrow bB'$$

$$B' \rightarrow \epsilon B' | \epsilon$$

Eliminate left recursion

$$S \rightarrow Aa | b$$

$$A \rightarrow Acl | sd | \epsilon$$

If we replace S by $Aa | b$ then

$$A \rightarrow Acl | Aad | bdl | \epsilon$$

if we replace S by $Aa | b$ then

Eliminate left recursion

$$S \rightarrow Aa | b$$

$$A \rightarrow Acl | sd | f$$

To eliminate the following rule will be apply:

$$A \rightarrow A\alpha_1 | A\alpha_2 | \beta_1 | \beta_2$$

then

$$A \rightarrow \beta_1 A' | \beta_2 A'$$

$$A' \rightarrow \alpha_1 A' | \alpha_2 A' | \epsilon$$

We replace S by $S \rightarrow Aa | b$

$$A \rightarrow A_c | A_d | b | f$$

⇒ After eliminating left recursion:

$$S \rightarrow A | b$$

$$A \rightarrow b | A' | f A'$$

$$A' \rightarrow c A' | a d A' | \epsilon$$

Q. Write rules to check grammar is left recursive or not. Remove left recursive from the following grammars

$$S \rightarrow a B D h$$

$$B \rightarrow B b | c$$

$$D \rightarrow E F$$

$$E \rightarrow g | \epsilon$$

$$F \rightarrow f | \epsilon$$

The rule of removal of left-recursion is as follows:

If $A \rightarrow A \alpha_1 | A \alpha_2 | B_1 | B_2$ is a left recursive rule then

$$A \rightarrow B_1 A' | B_2 A'$$

$$A' \rightarrow \alpha_1 A' | \alpha_2 A' | \epsilon$$

⇒ From given grammar, only one non-terminal B gives left recursive grammar

$$B \rightarrow B b | c$$

⇒ After eliminating left recursion:-

$$B \rightarrow C B'$$

$$B' \rightarrow b B' | \epsilon$$

To Summarize,

$$S \rightarrow a B D h$$

$$B \rightarrow C B'$$

$$B' \rightarrow b B' | \epsilon$$

$$D \rightarrow E F$$

$$E \rightarrow g | \epsilon$$

$$F \rightarrow f | \epsilon$$

Q. imp

Given $A \rightarrow s a | b$

$$S \rightarrow s c | A d$$

$$\Rightarrow A \rightarrow s c a | A d a | b$$

$$S \rightarrow s c | s a d | b d | b$$

Now apply left recursion

$$A \rightarrow b A'$$

$$A' \rightarrow c a A' | d a A' | \epsilon$$

$$S \rightarrow b d S' | a b d S'$$

$$S' \rightarrow c S' | a d S' | \epsilon$$

3. Ambiguity: (it is undecidable)

$\Rightarrow$ If more than one derivative trees exists for the given i/p string then that grammar has ambiguity.
 $\Rightarrow$ Derivative tree can be left-most or it can be right-most derivative tree.

$\Rightarrow$ The ambiguous grammar is not desirable in top-down parsing. Hence, we need to remove the ambiguity from the grammar if it is present.

Notes

- There is no standard method to check ambiguity. So it is undecidable.

- Problem with Ambiguity $\rightarrow$ Precedence of operators is violated / not respected.

- How to remove?

Note: If the grammar is left sensitive and right sensitive both then there will be ambiguity definitely.

$\Rightarrow$ The following rule:

To eliminate associativity problems:

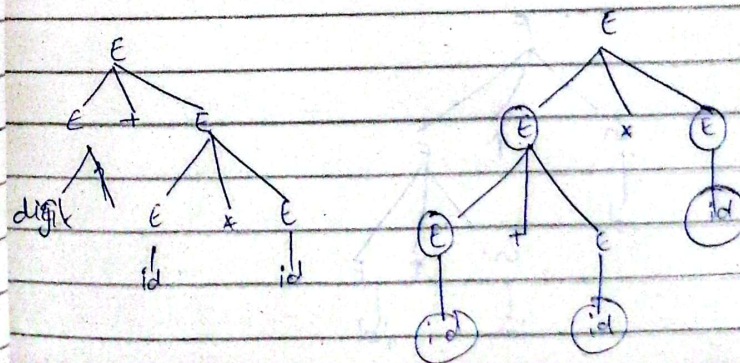
- If the grammar has left associative operators (such as $+$, $-$, $*$, $/$) then induce left association. To achieve left associativity we have to grow in $\swarrow$ left direction.
- If the operators in the grammar have right associativity (exponential operators) then induce the right association.

To eliminate precedence level:

- Highest operators should be at the least level.

Example of Ambiguous grammars:

$E \rightarrow E + E \mid E * E \mid id$ for $id + id * id$



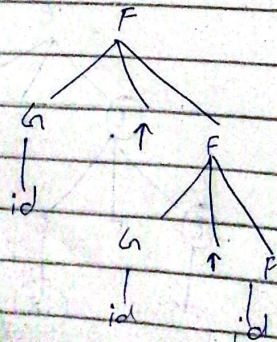
$F \rightarrow \text{id}$

2 ↑ 3 ↑ 2

$$(2 \uparrow (3 \uparrow 2))$$

$$F \xrightarrow{\quad} G \xrightarrow{F!/G} \text{signature}$$

$$G \xrightarrow{\quad} \text{id bit}$$



	Type	Operator
R-T-L	Unary	+, -, !, ~, ++, --, *, &, sizeof
	multiplicative	*, /, %
	Additive	+, -
L-T-R	Shift	<<, >>
	Relational	<, <=, >, >=
	Equality	==, !=
	Bitwise	&, , ^
	Logical	&&,
	Conditional	?:
R-T-L	Assignment	=, +=, -=, *=, /=, %=
	Comma	,

Precedence of logical:

OR } — Left Associative
AND }

NOT — Does not have associativity
Notice if the grammar is unambiguous
then $LMDT = RMDT$.

Q.

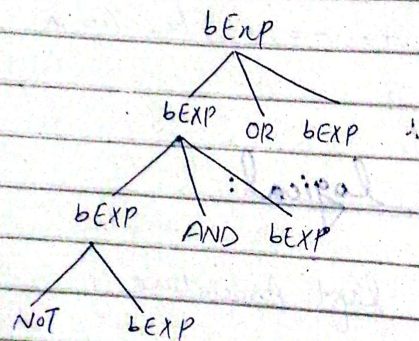
$bexp \rightarrow bexp \text{ OR } bexp / bexp$
 $/ \text{ NOT } bexp$
 $/ \text{ True } / \text{ false}$

To Convert Unambiguous:-

$E \rightarrow E \text{ OR } E / E$

$F \rightarrow F \text{ AND } G / G$

$G \rightarrow \text{NOT } G / \text{True} / \text{false}$



Q. Tell associativity & precedence of operators in the following grammar?

$A \rightarrow A \$ B / B$

$B \rightarrow B \# C / C$

$C \rightarrow C @ D / D$

$D \rightarrow d$

$\$, \#, @$ are left associative.

$@ > \# > \$ \rightarrow$ precedence.

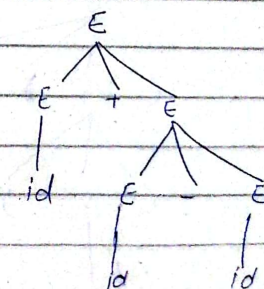
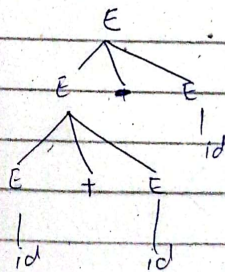
Q. Check grammar is ambiguous or not?

$E \rightarrow E + E$

$E \rightarrow E - E$

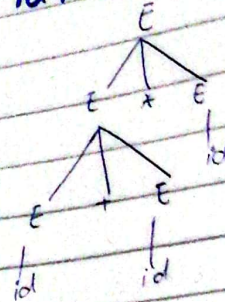
$E \rightarrow id$

s: $id + id - id$



Yes, the Grammar is ambiguous.

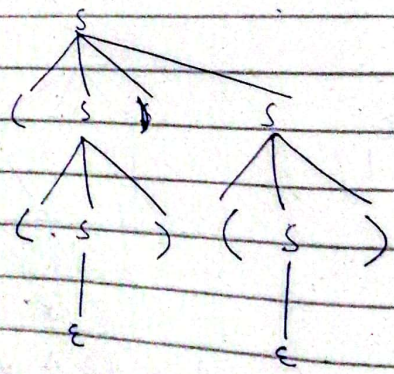
Q. Why the grammar failed:
 $id + id * id$



$\therefore$ Associativity failed.

Q. Check the grammar is ambiguous or not?
 $S \rightarrow (S)S$
 $S \rightarrow \epsilon$

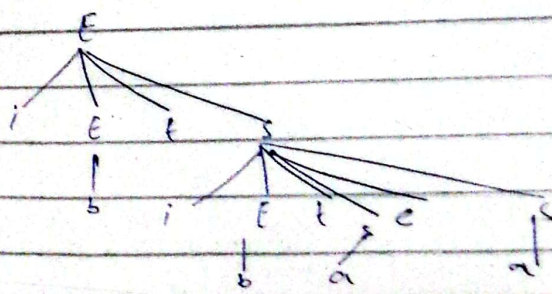
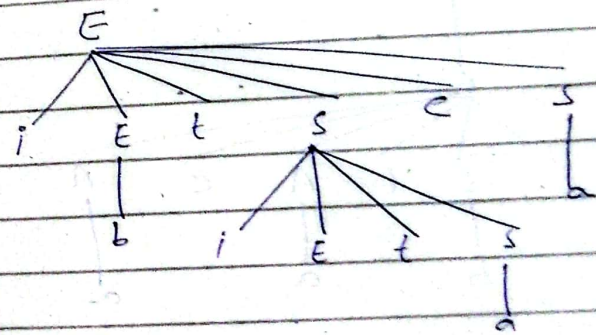
Consider the input string $((()))()$ for which we draw parse trees:



Q. Write: ambiguous and rules for if then else construct. Illustrate passing using both types of rules?

Ambiguous production rules for if-then-else:
 $S \rightarrow i E t S e S l i e t S l a$
 $E \rightarrow b$

Consider the string $ibtitaea$ can be passed.



Unambiguous grammar production rules
 if - then else

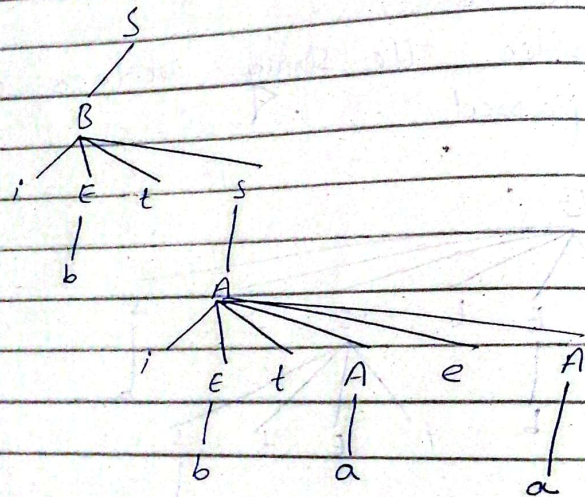
$S \rightarrow AIB$

$A \rightarrow iEtAeA|a$

$B \rightarrow iEtS|iEtAeB$

$E \rightarrow b$

The parsing of the string $iEtAeA$



Q. Generate parse tree from the given grammars.

Goal $\rightarrow$ Exp

Exp $\rightarrow$ Exp + term

| Exp - term

| term

term $\rightarrow$ term x value

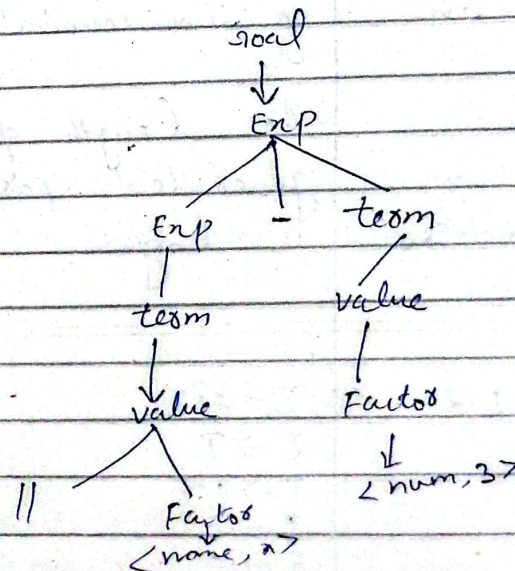
| term $\div$ value

| value

Value $\rightarrow$ // Factor | Factor

Factor $\rightarrow$ (Exp) | num | name

exp: // x-3



Q. Write difference between ambiguous and unambiguous grammar?

Ambiguous Grammar	unambiguous Grammar
(1) A grammar is said to be ambiguous if for at least one string generated by it, it produces more than one parse tree.	(1) A grammar is said to be unambiguous if for all the strings generated by it, it produces exactly one parse tree.
(2) In ambiguous grammar there are less number of non-terminals.	(2) In unambiguous grammar there are more number of non-terminals.
(3) The length of generated parse tree is less.	(3) The length of generated parse tree is large.
e.g.	e.g.
(4) $E \rightarrow E + E \mid E * E \mid id$	(4) e.g. $E \rightarrow E + T$ $E \rightarrow T$ $T \rightarrow T * F$ $T \rightarrow F$ $F \rightarrow id$

4. Left-factoring:-

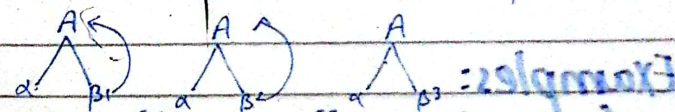
$\Rightarrow$ Left factoring is a technique used to remove or convert non-deterministic CFG to Deterministic CFG.

$\Rightarrow$ Top-down parsers may suffer from NCFG.

Non-deterministic CFG:

A CFG having more or one productions on the R.H.S are having something common in the prefixes. Also called common prefix problem.

e.g. $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3 \mid \dots \mid \alpha\beta_n$
for $\alpha\beta_1$



$\Rightarrow$ During top-down parsing non-deterministic grammar requires lots of backtracking (time consuming).

$\Rightarrow$ In top-down parsing we need to convert NDCFG into DCFG.

⇒ Left-factoring: **mainly** simplifies the grammar by introducing new symbols and rules, making parsing more efficient and reducing ambiguity.

In general it:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

is a production then it is not possible for us to take a decision whether to choose first rule or second. In such situation grammar can be left factored as:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow B_1 \mid B_2$$

Examples:

Consider the following CFN and do left-factoring.

$S \rightarrow iEts / iEtses / a$
 $E \rightarrow b$

$$E \rightarrow b$$

The subject of factored grammar becomes
S $\rightarrow$ IELTS '1a here as IELTS

$$S \rightarrow i E L S S' \mid a$$

$S' = \{a, b\}$

$$E \rightarrow h$$

Q. Do left-factoring in the following grammars?

$$A \rightarrow \alpha AB \mid \alpha A \mid \alpha$$

$$B \rightarrow bB/b$$

$A \rightarrow \alpha A B \mid \alpha A \mid \alpha$

$$1 \quad \alpha \quad \beta \quad \alpha \quad \beta$$

$$B \rightarrow \underline{6B} \mid L$$

$\alpha \quad \beta$

$$A \rightarrow \alpha A'$$

$$A' \rightarrow AB \mid A \mid \epsilon$$

$$B \rightarrow bB'$$

$$B' \rightarrow B \mid \varepsilon$$

Summarize:-

$$A \rightarrow \alpha A'$$

$$A' \rightarrow AB|A|E$$

$$B \rightarrow LB'$$

$$B' \rightarrow B/\epsilon$$

Q. Perform the left factoring on the following grammar.

$$A \rightarrow ad|a|ab|abc|b$$

Do left factoring:

$$A \rightarrow aA'|b$$

$$A' \rightarrow d|b|bc|\epsilon$$

$$A' \rightarrow d|b|bA'' \leftarrow A'$$

$$d|b|b \leftarrow b$$

Q.

$$S \rightarrow assbs|asab|abb|b$$

$$S \rightarrow as'|b$$

$$S' \rightarrow ssbs|sasb|bb|\epsilon$$

Still do left factoring on S'

$$S \rightarrow as'|b$$

$$S' \rightarrow ss''|bb|\epsilon$$

$$S'' \rightarrow sb|asb|\epsilon$$

Q. $E \rightarrow T+ET$
 $T \rightarrow \text{int}|\text{int} * T|(\epsilon)$

After doing left factoring

$$E \rightarrow TE'$$

$$E' \rightarrow +E|\epsilon$$

$$T \rightarrow \text{int} T' | (\epsilon)$$

$$T' \rightarrow *T|\epsilon$$

Q. What tools used for parsing?
 CFG

Q. Is it possible to make parse for every CFG?
 No, it is not possible.

Q. List down the classification of CFG?

